



Computer Networks

Ch.2: Application Layer

Prepared By
Dr. Ibrahim Attiya

© 2024 NMU

Ch.2: Outline

- ☐ Introduction
- ☐ Application-Layer Paradigms
- ☐ Client-Server Paradigm
- ☐ Socket Interface
- ☐ Standard Applications
- ☐ Peer-to-Peer Paradigm
- ☐ Socket Interface Programming
- ☐ Standard Client-Server Applications

World Wide Web

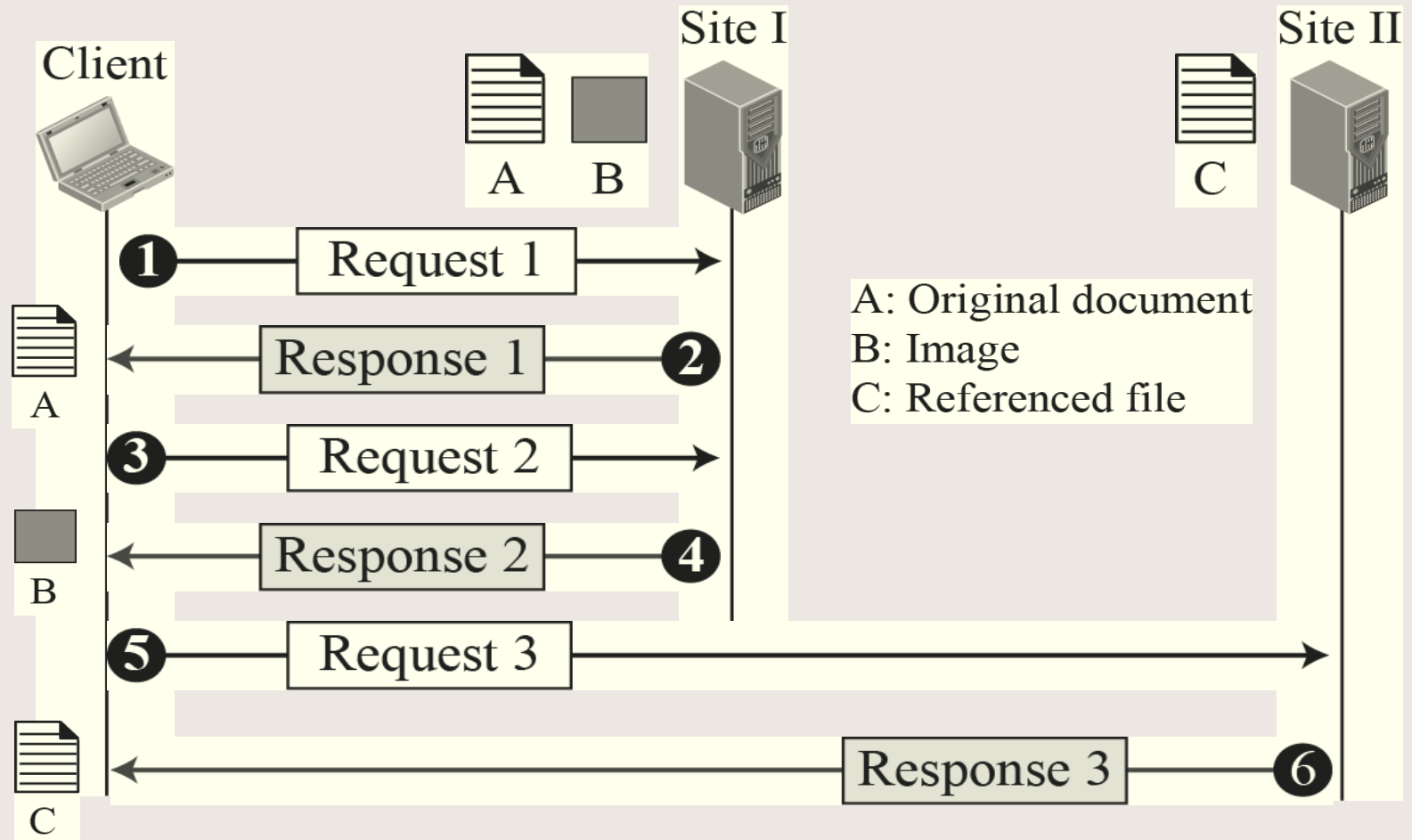
- The idea of the Web was first proposed by Tim Berners-Lee in 1989 at CERN, to allow several researchers at different locations throughout Europe to access each others' researches.
- The commercial Web started in the early 1990s.
- The Web today is a repository of information in which the documents, called *Web pages*, are distributed all over the world and related documents are linked together.

World Wide Web

Architecture

- The WWW today is a distributed client-server service, in which the service provided is distributed over many locations called *sites*.
- Each site holds one or more documents, referred to as web pages. Each **web page**, however, can contain some links to other web pages in the same or other sites.
- Each web page is a file with a name and address.
- A web page can be *simple* or *composite*.

World Wide Web



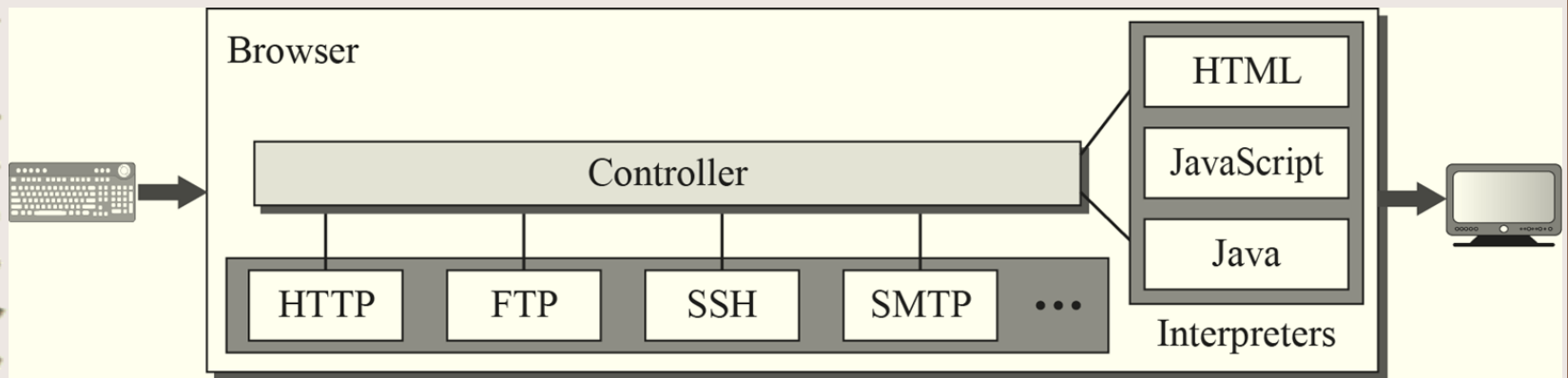
Retrieving two files and one image

World Wide Web

- **Web Client (Browser).** Web browsers are used to interpret and display a web page.
- Each browser usually consists of three parts: a *controller*, *client protocols*, and *interpreters*.
- The controller receives input from the keyboard or the mouse and uses the client programs to access the document.
- After the document has been accessed, the controller uses one of the interpreters to display the document on the screen.

World Wide Web

- The client protocol can be one of the protocols described later, such as *HTTP* or *FTP*.
- The interpreter can be *HTML*, *Java*, or *JavaScript*, depending on the type of document.
- Some commercial browsers include Internet Explorer, Netscape Navigator, and Firefox.



World Wide Web

- **Web Server.** The web page is stored at the server. Each time a request arrives, the corresponding document is sent to the client.
- To improve efficiency, servers normally store requested files in a *cache* in memory; memory is faster to access than disk.
- A server can also become more efficient through *multithreading* or *multiprocessing*.
- Some popular web servers include Apache and Microsoft Internet Information Server.

HyperText Transfer Protocol (HTTP)

- The HTTP is a protocol that is used to define how the client-server programs can be written to retrieve web pages from the Web.
- An HTTP client sends a request; an HTTP server returns a response. The client and server use the port number 80.
- HTTP uses the services of TCP, which, as discussed before, is a *connection-oriented* and *reliable* protocol.

HyperText Transfer Protocol (HTTP)

- Before any transaction between the client and the server can take place, a connection needs to be established between them.
- After the transaction, the connection should be terminated.
- The client and server, however, do not need to worry about errors in messages exchanged or loss of any message, because the TCP is reliable and will take care of this matter.

Nonpersistent versus Persistent Connections

- If the web pages, objects to be retrieved, are located on different servers, we have to create a new TCP connection for retrieving each object.
- However, if some of the objects are located on the same server, we have two choices: to retrieve each object using a new TCP connection or to make a TCP connection and retrieve them all.
 - The first method is referred to as *nonpersistent connections*, the second as *persistent connections*.

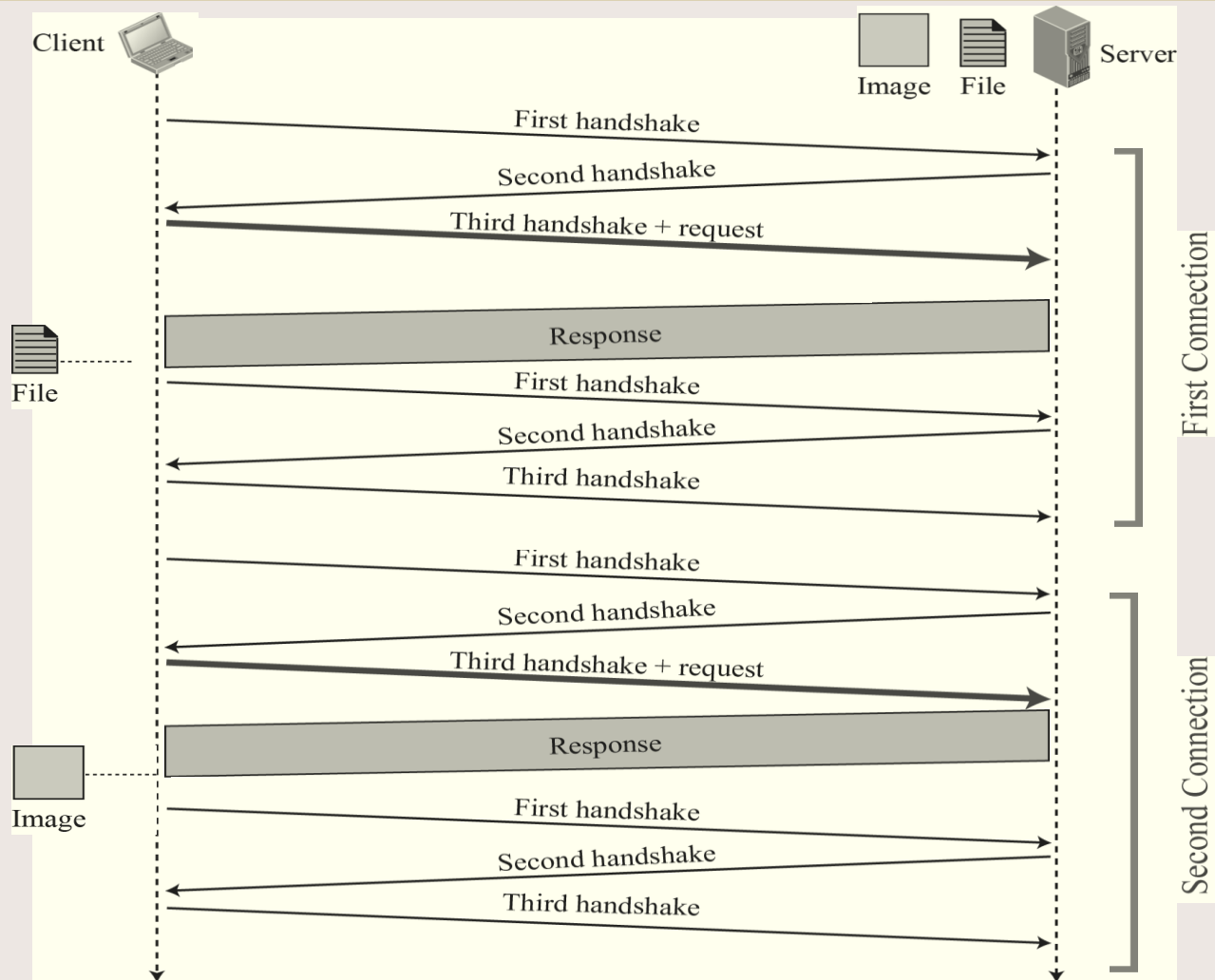
Nonpersistent Connections

- In a *nonpersistent connection*, one TCP connection is made for each request/response.
- The following lists the steps in this strategy:
 1. The client opens a TCP connection and sends a request.
 2. The server sends the response and closes the connection.
 3. The client reads the data until it encounters an end-of-file marker; it then closes the connection.

Nonpersistent Connections

- In this strategy, if a file contains links to N different pictures in different files (all located on the same server), the connection must be opened and closed $N + 1$ times.
- The nonpersistent strategy imposes *high overhead* on the server because the server needs $N + 1$ different buffers each time a connection is opened.

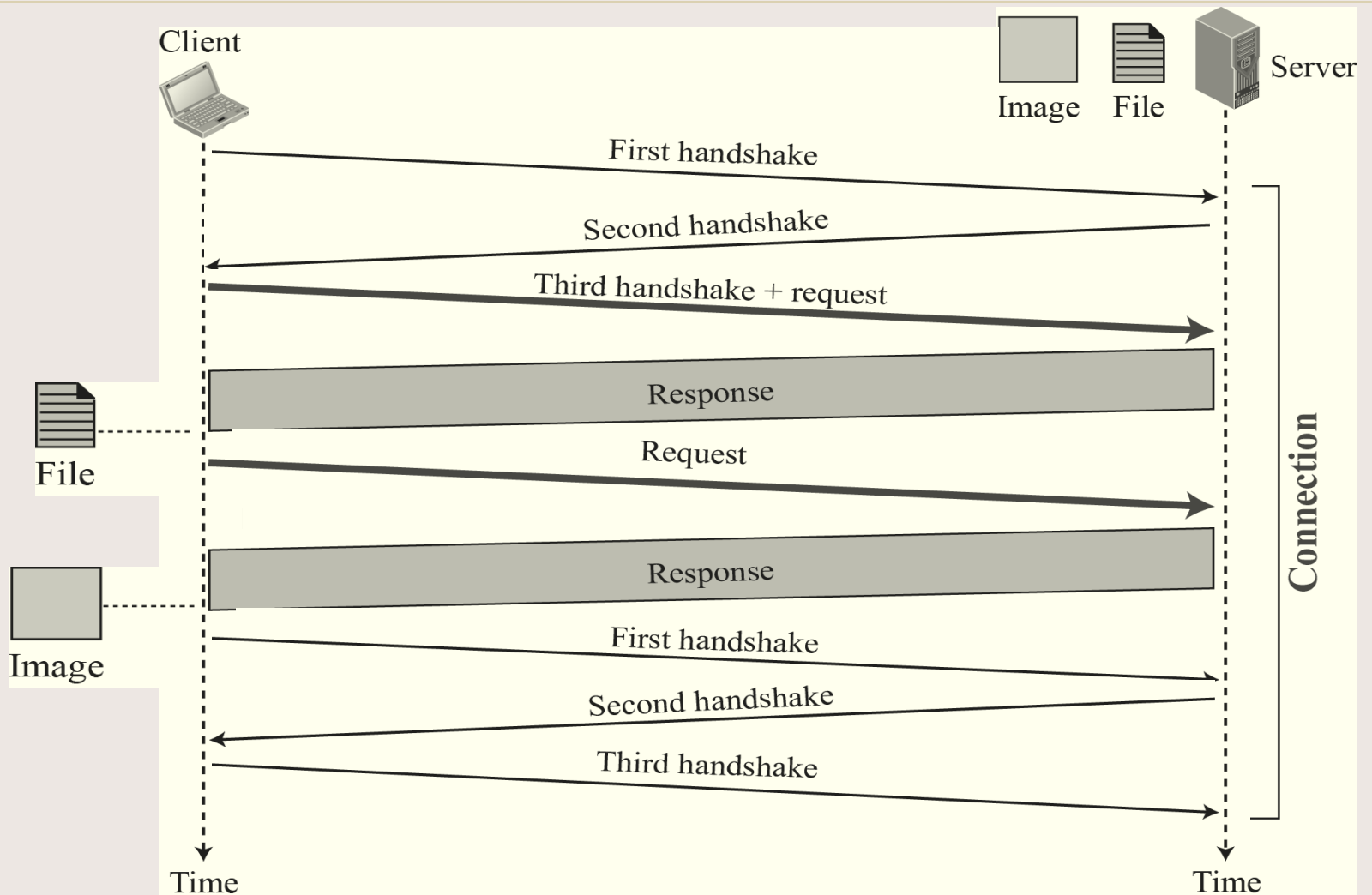
Nonpersistent Connections



Persistent Connections

- HTTP version 1.1 specifies a persistent connection by default.
- In a *persistent connection*, the server leaves the connection open for more requests after sending a response.
- The server can close the connection at the request of a client or if a time-out has been reached.
- Time and resources are saved using persistent connections. The *round-trip time* for connection establishment and connection termination is saved.

Persistent Connections



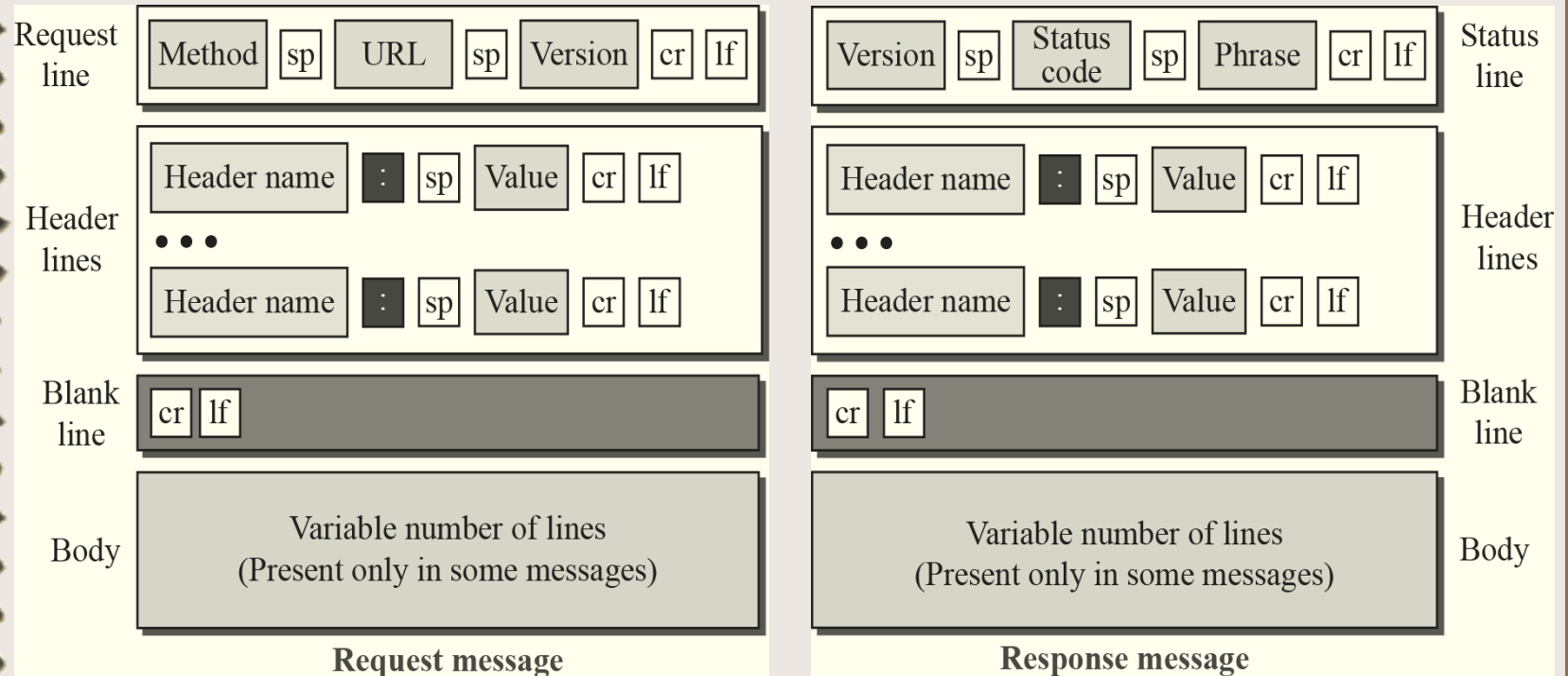
HyperText Transfer Protocol (HTTP)

Message Formats

- The HTTP protocol defines the format of the request and response messages. Each message is made of four sections.
 - The first section in the request message is called the *request line*; the first section in the response message is called the *status line*.
 - The other three sections have the same names in the request and response messages. However, the similarities between these sections are only in the names; they may have different contents.

HyperText Transfer Protocol (HTTP)

Legend sp: Space cr: Carriage Return lf: Line Feed



Formats of the request and response messages

HyperText Transfer Protocol (HTTP)

Request Message

- As we said before, the first line in a request message is called a *request line*.
- There are three fields in the request line separated by one space and terminated by two characters (carriage return and line feed).
- The fields are called *method*, *URL*, and *version*.
- The *method* field defines the request types.
- In version 1.1 of HTTP, several methods are defined, as shown in the next slides.

HyperText Transfer Protocol (HTTP)

- The *GET* method is used when the browser requests an object, with the requested object identified in the URL field.
- The *HEAD* method is used when the client needs only some information about the web page from the server, such as the last time it was modified.
- The *PUT* method is the inverse of the GET method; it allows the client to post a new web page on the server (if permitted).

HyperText Transfer Protocol (HTTP)

- The *POST* method is similar to the *PUT* method, but it is used to send some information to the server to be added to the web page or to modify the web page.
- The *TRACE* method is used for debugging; the client asks the server to echo back the request to check whether the server is getting the requests.
- The *DELETE* method allows the client to delete a web page on the server if the client has permission to do so.

HyperText Transfer Protocol (HTTP)

- The *CONNECT* method was originally made as a reserve method; it may be used by proxy servers.
- Finally, the *OPTIONS* method allows the client to ask about the properties of a web page.

<i>Method</i>	<i>Action</i>
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

HyperText Transfer Protocol (HTTP)

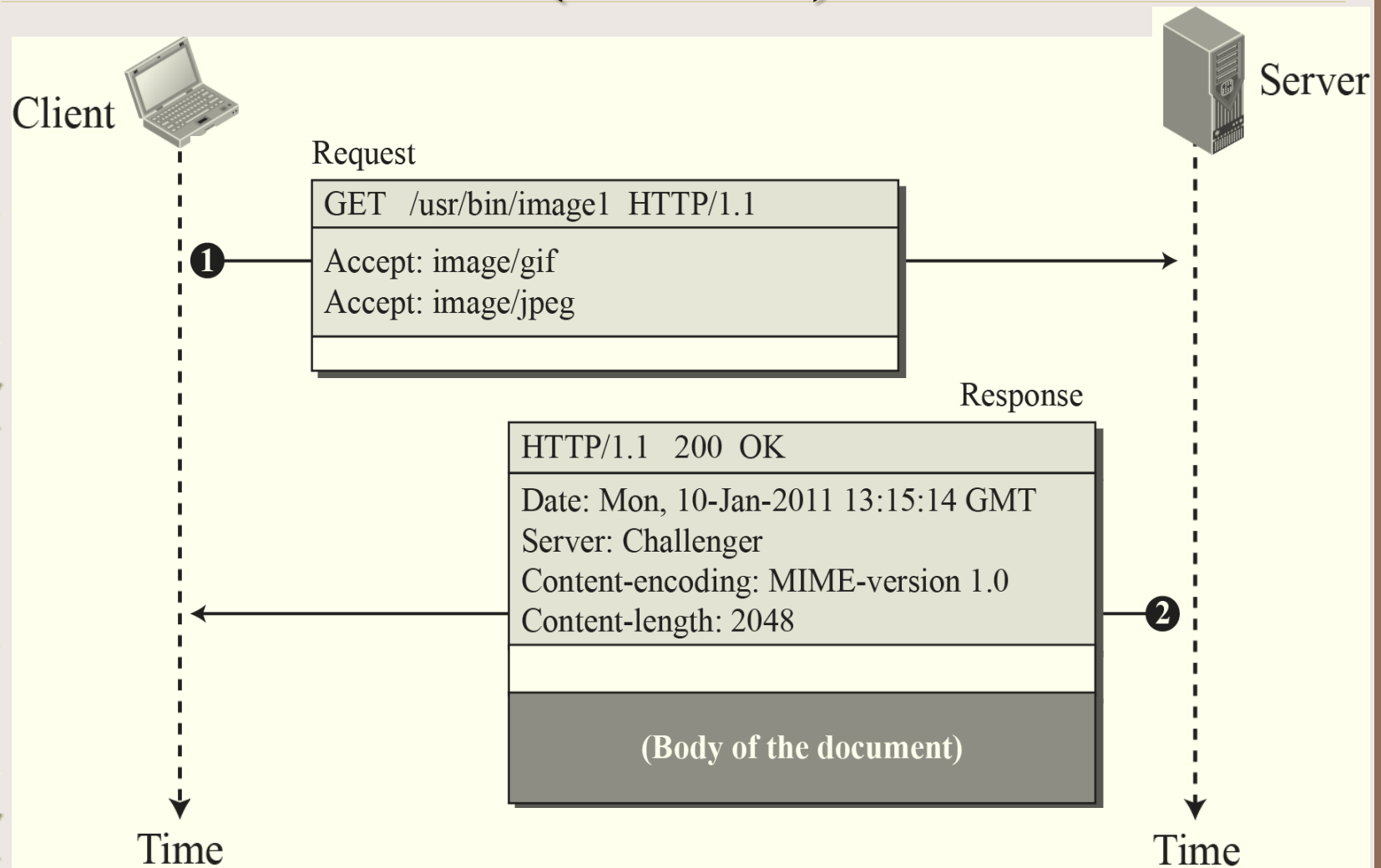
Response Message

- A response message consists of a *status line*, *header lines*, a *blank line*, and sometimes a *body*.
- The first line in a response message is called the *status line*. There are three fields in this line separated by spaces and terminated by a carriage return and line feed.
- The first field defines the version of HTTP protocol.

HyperText Transfer Protocol (HTTP)

- The status code field consists of three digits:
 - The codes in the 100 range are only informational.
 - The codes in the 200 range indicate a successful request.
 - The codes in the 300 range redirect the client to another URL.
 - The codes in the 400 range indicate an error at the client site.
 - Finally, the codes in the 500 range indicate an error at the server site.

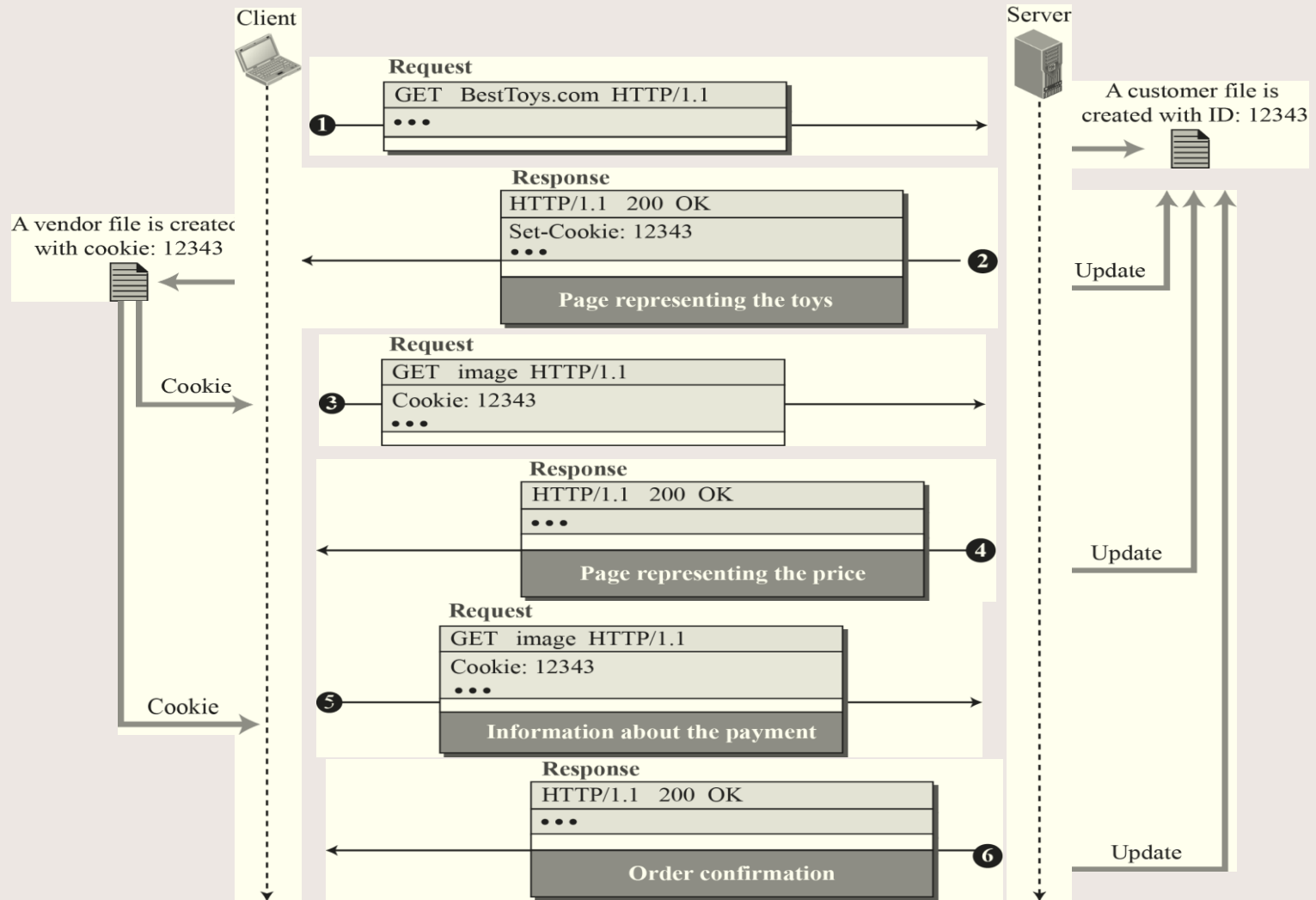
HyperText Transfer Protocol (HTTP)



Cookies

- Today the Web has other functions that need to remember some information about the clients; some are listed below:
 - Websites are being used as *electronic stores* that allow users to browse through the store, etc.
 - Some websites need to allow access to *registered clients* only.
 - Some websites are used as *portals*: the user selects the web pages he wants to see.
 - Some websites are just *advertising* agency.
- For these purposes, the cookie mechanism was devised.

Cookies



Cookies

Creating and Storing Cookies.

- The creation and storing of cookies depend on the implementation; however, the principle is the same.
 1. When a server receives a request from a client, it stores information about the client in a file or a string.
 2. The server includes the cookie in the response that it sends to the client.
 3. When the client receives the response, the browser stores the cookie in the cookie directory, which is sorted by the server domain name.

Cookies

Using Cookies

- When a client sends a request to a server, the browser looks in the cookie directory to see if it can find a cookie sent by that server.
- If found, the cookie is included in the request.
- When the server receives the request, it knows that this is an old client, not a new one.
- Note that the contents of the cookie are never read by the browser or disclosed to the user. It is a cookie *made* by the server and *eaten* by the server.

Cookies: Summary

- Cookies are small files of information generated by a web server while a user is browsing a website and placed on the user's device by the user's web browser.
- *What cookies can be used for?*
 - Authorization
 - Shopping carts
 - Recommendations
 - User session state.
- Although cookies often simplify the Internet shopping experience for the user, they are controversial because they can also be considered as an invasion of privacy.

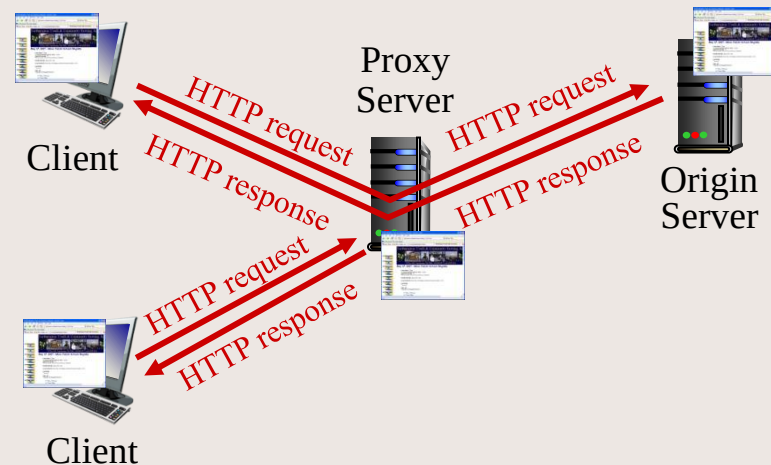
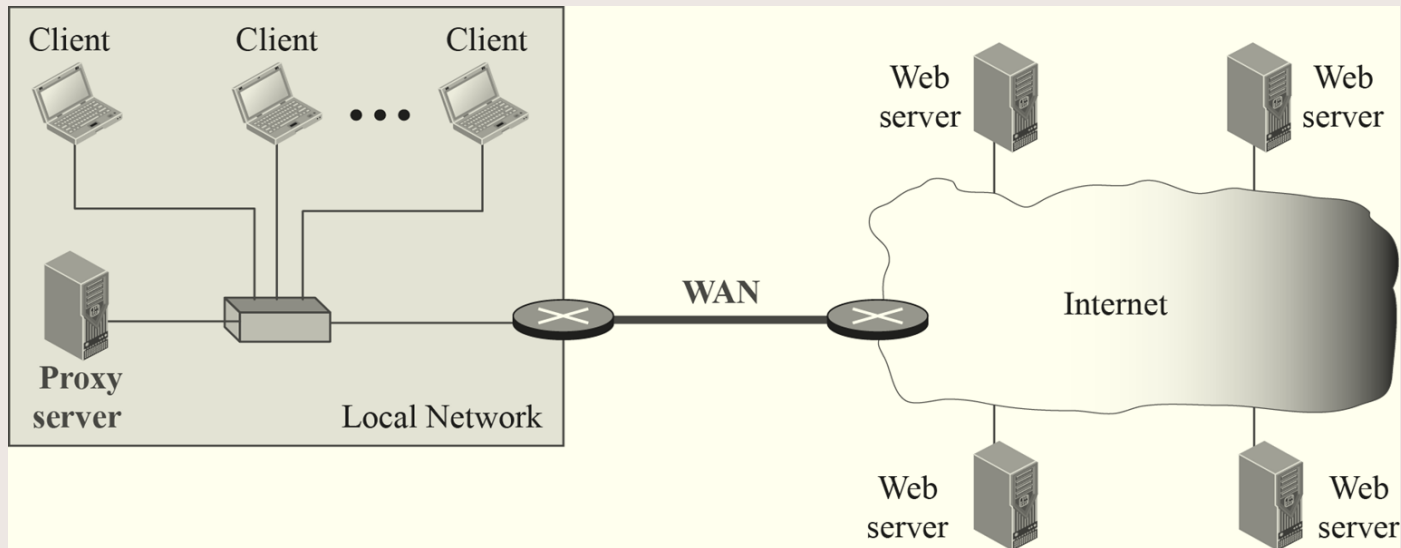
Web Caching: Proxy Server

- A ***proxy server*** is a computer that keeps copies of responses to recent requests.
- HTTP supports ***proxy servers***.
 - The HTTP client sends a request to the proxy server. The proxy server checks its cache.
 - If the response is not stored in the cache, the proxy server sends the request to the corresponding server.
 - Incoming responses are sent to the proxy server and stored for future requests from other clients.
- The proxy server *reduces the load on the original server, decreases traffic, and improves latency.*

Web Caching: Proxy Server

- Note that the proxy server acts as both server and client.
 - Server for original requesting client.
 - Client to origin server.
- However, to use the proxy server, the client must be configured to access the proxy instead of the target server.
- Typically, cache is installed by ISP (university, company, residential ISP).

Web Caching: Proxy Server



Web Caching: Proxy Server

Proxy Server Location

- The proxy servers are normally located at the client site.
 - A client computer can also be used as a proxy server, in a small capacity, that stores responses to requests often invoked by the client.
 - In a company, a proxy server may be installed on the computer LAN to reduce the load going out of and coming into the LAN.
 - An ISP with many customers can install a proxy server to reduce the load going out of and coming into the ISP network.

Web Caching: Proxy Server

Cache Update

- How long a response should remain in the proxy server before being deleted and replaced?
 - One solution is to store the list of sites whose information remains the same for a while.
 - Another recommendation is to add some headers to show the last modification time of the information. The proxy server can then use the information in this header to guess how long the information would be valid.

From HTTP/1.1 to HTTP/3

- **HTTP1.1**: introduced *multiple, pipelined GETs* over a single TCP connection.
 - The server responds *in-order* (FCFS scheduling) to GET requests.
- **HTTP/2**: increased flexibility at *server* in sending objects to client.
 - Transmission order of requested objects based on client-specified object priority (not necessarily FCFS).
- **HTTP/3**: adds security, per object error- and congestion-control (more pipelining) over UDP.
- **HTTP** can be run over the *Secure Socket Layer (SSL)*. In this case, HTTP is referred to as **HTTPS**.
- **HTTPS** provides *confidentiality*, client and server *authentication*, and *data integrity*.

A silver-colored metal spiral binding is visible along the left edge of the page, consisting of a series of loops.

Any Questions?